

RTEMS에서 개선된 일시적 실행 연장을 통한 마이그레이션 최소화 및 우선순위 역전 문제 완화 방안 연구

Minimizing Migration and Mitigating Priority Inversion
Using Temporal Boosting on RTEMS

옥지민

(인하대학교 컴퓨터공학과 학부생)

정진만

(인하대학교 컴퓨터공학과 교수)

Key Words : RTEMS, Hard Real-Time OS, Real-Time Scheduler, EDF, Temporal Boosting, Priority Inversion

목 차

서론
관련 연구
제안 기법
실험 및 평가
결론
참고 문헌

I. 서론

RTEMS(Real-Time Executive for Multiprocessor Systems)는 오픈소스 기반 실시간 운영체제로 항공 및 우주 분야에서 실시간성을 보장한다. 그러나 기존의 RTEMS의 EDF(Earliest Deadline First) 스케줄러는 잦은 선점으로 인해 마이그레이션(migration)이 발생하여 성능 저하가 발생할 수 있다. 또한 자원을 소유한 채로 선점될 경우, 우선순위 역전(Priority Inversion) 문제를 심화시킬 수 있다. 본 연구는 RTEMS의 스케줄링 구조를 분석하고 특정 조건을 만족하는 경우, 현재 스레드의 선점을 지연시켜 기존 코어에서의 실행을 보장해 임계 구역을 조기에 완료하고 자원을 신속하게 해제하도록 유도하여 우선순위 역전 문제를 완화하는 것을 목표로 한다.

II. 관련 연구

EDF 스케줄러는 동적인 방식으로 스케줄링 하여 선점이 불필요하게 많이 일어날 수 있다. 심지어 다중코어 환경에서는 선점이 자주 발생하게 되면 선점된 스레드를 다른 코어로 보내야 할 수 있으므로 마이그레이션이 발생하여 캐시 무효화(Cache Invalidation)가 발생할 수 있다.

이를 해결하기 위한 다양한 연구들이 진행되었다.

- Preemption Threshold Scheduling(PTS) [1]: 태스크별 선점 임계값을 설정하여 불필요한 선점을 줄이는 기법이다.
- Limited Preemption EDF Scheduling [2]: EDF의 선점형

특성은 유지하면서 불필요한 선점은 억제하여 오버헤드를 줄이는 하이브리드 방식이다.

III. 제안 기법

1. Temporal Boosting의 개념

Temporal Boosting은 현재 스레드가 계속 실행될 경우 예상되는 종료 시각이 다음 스레드가 마감기한을 맞출 수 있는 “최소 시작 시간”(Least Feasible Start Time)보다 이르면, 다음 스레드의 선점을 지연시켜 현재 스레드의 실행시간을 보장해주는 기법이다.

Temporal Boosting은 다음 조건을 만족할 때 허용된다.

$$T_{cur} + R_{cur} < D_{next} - R_{next} - G$$

여기서

T_{cur} 은 현재 시간(current time),

R_{cur} 은 현재 스레드의 남은 실행 시간(remain time), D_{next} 은

다음 스레드의 마감기한(deadline),

R_{next} 은 다음 스레드의 남은 실행시간(remain time),

G 는 Guard Time으로서 예외 상황을 대비하기 위한 상수 시간이다.

이를 통해 불필요한 선점을 방지하고, 기존 코어에서 연장된 실행시간을 보장하여 임계 구역을 신속하게 해제하도록 유도

해 우선순위 역전 문제를 완화하는 것이 목표이다.

2. RTEMS 확장을 통한 TCB 확장

제안 기법을 구현하기 위해 RTEMS의 사용자 확장(user extension) 기능을 통해 `edf_extension`을 추가하였다. 사용자 확장 기능은 RTEMS C User's Guide [3]을 참고하였다. `edf_extension`은 TCB(Task Control Block)에 Temporal Boosting에 필요한 추가 데이터를 저장하는 구조체이다.

<edf_extension.h>

```
typedef struct{
    bool isAdvanced;
    uint64_t remain_time;
    uint64_t execution_time;
    uint64_t latest_scheduled_time;
    uint64_t deadline;
} edf_thread_data;
```

여기서 `isAdvanced`는 스케줄링 시 Temporal Boosting을 허용하는 스레드인지 여부를 나타내며 `remain_time`과 `execution_time`은 스레드의 진행 상태를 추적하기 위해 사용된다.

`latest_scheduled_time`은 남은 시간을 관리하기 위해 사용되며 `deadline`은 스레드의 절대 마감 시각을 의미한다.

3. 실행시간 및 주기 관리

각 스레드 생성 후, `edf_set_execution_time()` 함수를 통해서 총 실행시간과 이번 주기의 마감시각을 설정한다.

RTEMS의 `edf`는 주기적 태스크를 대상으로 하기 때문에 새 주기가 시작되면 `deadline`과 `execution time`을 `edf_start_period()` 함수를 통해 새 주기에 맞게 설정해 준다. `edf_start_period()` 함수는 RTEMS의 Monotonic Manager 내부에서 호출되게 설계하여 호환성을 높였다.

4. 남은 작업 시간 관리

RTEMS의 `_Thread_Do_dispatch.c` 내부의 선점이 발생하는 부분에서 현재 실행 중인 스레드의 남은 시간을 갱신하도록 설계하였다.

```
// in void _Thread_Do_dispatch(...)
if(...){
    uint64_t work_time
        = cur_time
        - cur_data->latest_scheduled_time;

    if(cur_data->remain_time > work_time)
        cur_data->remain_time -= work_time;
    else
        cur_data->remain_time = 0;

    cur_data->latest_scheduled_time
        = cur_time;
}
```

이 방식은 매 타이머 인터럽트마다 `remain_time`을 감소시키는 방법보다 시스템 비용 측면에서 효율적이다.

5. Temporal Boosting 판단

Boosting 여부는 현재 스레드의 예상 종료 시각(`cur_fin_time`) 이 다음 스레드가 마감 기한을 맞추기 위해 최소한으로 시작해야 하는 시각(Least Feasible Start Time)보다 앞선 경우에 한해 허용한다. 즉, 현재 스레드가 좀 더 실행되어도 다음 스레드에 마감기한에 영향을 주지 않는다면, 현재 스레드의 작업 시간을 잠시 연장(Temporal Boosting)한다.

```
// in void _Thread_Do_dispatch(...)
if(...){
    uint64_t cur_fin_time
        = cur_time + cur_data->remain_time;
    bool isBoost
        = ... && (cur_fin_time
        < (next_data->deadline
        - next_data->remain_time - 10));
    if(isBoost){
        ...
        break; // BOOSTING
    }
}
if(next_data && next_data->isAdvanced)
    next_data->latest_scheduled_time
        = cur_time;
```

이때, 10은 실험적으로 설정한 Guard Time이며 예상치 못한 예외로 인한 deadline miss를 방지하는 역할을 한다.

IV. 실험 및 평가

본 연구에서는 RTEMS 환경에서 Temporal Boosting 기법의 성능을 평가하였다. 실험은 sparc sis 시뮬레이터 환경을 사용하였다. 테스트 대상은 LEON3 프로세서를 에뮬레이션한 단일 코어(UP) 환경에서 진행하였다.

1. 실험환경

운영체제: RTEMS-6

시뮬레이터: SIS(SPARC/LEON3)

2. 테스트 시나리오

테스트는 2개의 주기적 태스크(Periodic Task) A와 B로 구성하였다.

<표 1>

태스크	주기	실행시간	등장시간
A	200	30	0
B	100	32	10

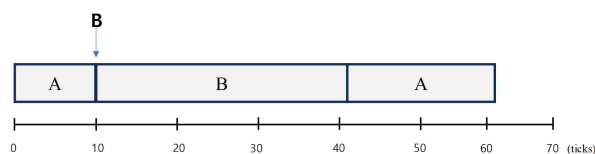
<표 1>은 본 연구에서 사용된 주기적 태스크 A, B의 주기, 실행시간 그리고 등장시간을 나타낸다.

3. 테스트 결과

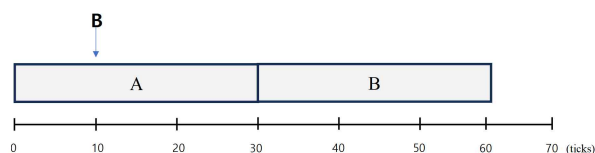
<표 2>

작업량	Boosting 적용 시 context switch 수	Boosting 미적용 시 context switch 수
200	499	600
400	999	1200
800	1999	2400
1600	3999	4800

Boosting을 적용한 경우, 문맥 전환 횟수가 약 16.7 % 감소하였으며, 이는 스케줄링 오버헤드 감소 효과를 보여준다.



<그림 1>



<그림 2>

<그림 1>은 Temporal Boosting이 미적용된 경우 스케줄링 첫 주기의 타임라인을 나타내며 <그림 2>는 Temporal Boosting이 적용된 경우를 나타낸다.

<그림 2>의 경우, <그림 1>과 비교하여 불필요한 문맥 전환이 감소하였고 마감 기한 또한 지킨 것을 확인할 수 있다.

4. 결과 분석

Temporal Boosting은 스레드가 남은 실행 시간을 마치기 전에 불필요하게 선점되지 않도록 한다. 이는 문맥 전환을 줄여 연장된 실행시간을 보장함으로써 스레드가 임계영역을 신속하게 탈출할 수 있게 해준다. 그 결과, 우선순위 역전과 더불어 MP 환경에서는 마이그레이션이 감소하는데 기여할 것으로 기대된다.

V. 결론

본 연구에서는 RTEMS 환경에서 Temporal Boosting 기법을 제안하고 그 유효성을 검증하였다. 실험 결과, 제안 기법은 불필요한 문맥 전환(Context Switch)을 약 16.7% 감소시켰으며, 이는 스케줄링 오버헤드를 효과적으로 줄일 수 있음을 입증한다. 이러한 선점 빈도의 감소는 본 논문에서 제기한 마이그레이션 발생 가능성과 우선순위 역전 문제를 구조적으로 완화할 수 있는 잠재력을 보여준다. 향후에는 본 기법을 멀티프로세서(SMP) 환경으로 확장 적용하여, 실제 마이그레이션 억제 효과와 우선순위 역전 완화 정도를 정량적으로 분석하는 연구를 수행할 계획이다.

참고문헌

- [1] Y. Wang and M. Saksena, "Scheduling Fixed-Priority Tasks with Preemption Threshold," in Proc. IEEE Int. Conf. Real-Time Comput. Syst. Appl. (RTCSA), pp. 328-335, 1999.
- [2] M. Bertogna and S. Baruah, "Limited Preemption EDF Scheduling of Sporadic Task Systems," IEEE Transactions on Industrial Informatics, vol. 6, no. 4, pp. 579-591, Nov. 2010. doi: 10.1109/TII.2010.2049654
- [3] The RTEMS Project, RTEMS C User's Guide (Version 4.11.0), RTEMS Documentation. Available: <https://docs.rtems.org/docs/4.11.0/c-user/index.html>